



Dependency Injection

Ha Noi Aug - 2023

OPEN MIND - OPEN DOORS



Table of content

Dependency injection in Android	3
Dagger-Hilt	3
Koin	3

OPEN MIND - OPEN DOORS

Dependency Injection in Android

OPEN MIND - OPEN DOORS

Overview DI

- Dependency injection (DI) is a technique widely used in programming and well suited to Android development. By following the principles of DI, you lay the groundwork for good app architecture.
- Advantage:
 - Reusability of code
 - Ease of refactoring
 - Ease of testing

What is Dependency Injection

- Classes often require references to other classes.
- There are three ways for a class to get an object it needs:
 - The class constructs the dependency it needs. In the example above, `Car` would create and initialize its own instance of `Engine`.
 - Grab it from somewhere else. Some Android APIs, such as `Context` getters and `getSystemService()`, work this way. Have it supplied as a parameter. The app can provide these dependencies when the class is constructed or pass them in to the functions that need each dependency. In the example above, the `Car` constructor would receive `Engine` as a parameter.
- The third option is dependency injection! With this approach you take the dependencies of a class and provide them rather than having the class instance obtain them itself.

What is Dependency Injection

- There are two major ways to do dependency injection in Android:
 - **Constructor Injection.** This is the way described above. You pass the dependencies of a class to its constructor.
 - **Field Injection (or Setter Injection).** Certain Android framework classes such as activities and fragments are instantiated by the system, so constructor injection is not possible. With field injection, dependencies are instantiated after the class is created. The code would look like this:

Dagger - Hilt

OPEN MIND - OPEN DOORS

Hilt Dependency Injection

- Hilt is a dependency injection library for Android that reduces the boilerplate of doing manual dependency injection in your project. Doing [manual dependency injection](#) requires you to construct every class and its dependencies by hand, and to use containers to reuse and manage dependencies.
- Hilt provides a standard way to use DI in your application by providing containers for every Android class in your project and managing their lifecycles automatically. Hilt is built on top of the popular DI library [Dagger](#) to benefit from the compile-time correctness, runtime performance, scalability, and [Android Studio support](#) that Dagger provides. For more information, see [Hilt and Dagger](#).

```
buildscript {  
    ...  
    ext.hilt_version = '2.38.1'  
    dependencies {  
        ...  
        classpath "com.google.dagger:hilt-android-gradle-plugin:$hilt_version"  
    }  
}
```

```
plugins {  
    kotlin("kapt")  
    id("dagger.hilt.android.plugin")  
}  
  
android {  
    ...  
}  
  
dependencies {  
    implementation("com.google.dagger:hilt-android:$hilt_version")  
    kapt("com.google.dagger:hilt-android-compiler:$hilt_version")  
}
```

Hilt Dependency Injection

Hilt & Dagger Annotations

v2.33 - @AndroidDev

Annotation	Usage	Code Sample
@HiltAndroidApp	Kicks off Hilt code generation. Must annotate the Application class.	<pre>@HiltAndroidApp class MyApplication : Application() { ... }</pre>
@AndroidEntryPoint	Adds a DI container to the Android class annotated with it. This requires using Hilt's Gradle Plugin.	<pre>@AndroidEntryPoint class MyActivity : AppCompatActivity() { ... }</pre>
@Inject	Constructor Injection. Tells which constructor to use to provide instances and which dependencies the type has.	<pre>class AnalyticsAdapter @Inject constructor(private val service: AnalyticsService) { ... }</pre>
	Field injection. Populates fields in @AndroidEntryPoint annotated classes. Fields cannot be private.	<pre>@AndroidEntryPoint class MyActivity : AppCompatActivity() { @Inject lateinit var adapter: AnalyticsAdapter ... }</pre>
@HiltViewModel	Tells Hilt how to provide instances of an Architecture Component ViewModel.	<pre>@HiltViewModel class MyViewModel @Inject constructor(private val adapter: AnalyticsAdapter, private val state: SavedStateHandle): ViewModel() { ... }</pre>
@Module	Class in which you can add bindings for types that cannot be constructor injected.	<pre>@InstallIn(SingletonComponent::class) @Module class AnalyticsModule { ... }</pre>

Hilt Dependency Injection

Hilt & Dagger Annotations

v2.33 - @AndroidDev

Annotation	Usage	Code Sample
@InstallIn	Indicates in which Hilt-generated DI containers (SingletonComponent in the code) module bindings must be available.	<pre><code>@InstallIn(SingletonComponent::class) @Module class AnalyticsModule { ... }</code></pre>
@Provides	Adds a binding for a type that cannot be constructor injected: - Return type is the binding type. - Parameters are dependencies. - Every time an instance is needed, the function body is executed if the type is not scoped.	<pre><code>@InstallIn(SingletonComponent::class) @Module class AnalyticsModule { @Provides fun providesAnalyticsService(converterFactory: GsonConverterFactory): AnalyticsService { return Retrofit.Builder() .baseUrl("https://example.com") .addConverterFactory(converterFactory) .build() .create(AnalyticsService::class.java) } }</code></pre>
@Binds	Shorthand for binding an interface type: - Methods must be in a module. - @Binds annotated methods must be abstract. - Return type is the binding type. - Parameter is the implementation type.	<pre><code>@InstallIn(SingletonComponent::class) @Module abstract class AnalyticsModule { @Binds abstract fun bindsAnalyticsService(analyticsServiceImpl: AnalyticsServiceImpl): AnalyticsService }</code></pre>
Scope Annotations: @Singleton @ActivityScoped ...	Scoping object to a container. The same instance of a type will be provided by a container when using that type as a dependency, for field injection, or when needed by containers below in the hierarchy.	<pre><code>@Singleton class AnalyticsAdapter @Inject constructor(private val service: AnalyticsService) { ... }</code></pre>

OPEN MIND - OPEN DOORS

Hilt Dependency Injection

Hilt & Dagger Annotations		
v2.33 - @AndroidDev		
Annotation	Usage	Code Sample
Qualifiers for predefined Bindings: @ApplicationContext @ActivityContext	Predefined bindings you can use as dependencies in the corresponding container. These are qualifier annotations.	<pre> @Singleton class AnalyticsAdapter @Inject constructor(@ApplicationContext val context: Context private val service: AnalyticsService) { ... } </pre>
@Entry Point	Obtain dependencies in classes that are either not supported directly by Hilt or cannot use Hilt. The interface annotated with @EntryPoint must also be annotated with @InstallIn passing in the Hilt predefined component from which that dependency is taken from. Access the bindings using the appropriate static method from EntryPointAccessors passing in an instance of the class with the DI container (which matches the value in the @InstallIn annotation) and the entry	<pre> class MyContentProvider(): ContentProvider { @InstallIn(SingletonComponent::class) @EntryPoint interface MyContentProviderEntryPoint { fun analyticsService(): AnalyticsService } override fun query(...): Cursor { val appContext = context?.applicationContext ?: throw IllegalStateException() val entryPoint = EntryPointAccessors.fromApplication(appContext, MyContentProviderEntryPoint::class.java) val analyticsService = entryPoint.analyticsService() ... } } </pre>
For more information about DI, Hilt and Dagger: https://d.android.com/dependency-injection		

OPEN MIND - OPEN DOORS

Room Database with Hilt

This example we create a database and dao with hilt, we also inject a coroutine scope to insert fake data in first time database created

```
@InstallIn(SingletonComponent::class)
@Module
object CoroutinesScopesModule {

    @Singleton // Provide always the same instance
    @Provides
    fun providesCoroutineScope(): CoroutineScope {
        // Run this code when providing an instance of CoroutineScope
        return CoroutineScope(context: SupervisorJob() + Dispatchers.IO)
    }
}
```

```
@HiltViewModel
class TaskViewModel @Inject constructor(
    private val taskRepository: TaskRepository
) : ViewModel() {
```

```
@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {
    @Provides
    @Singleton
    fun provideTaskDatabase(
        application: Application,
        scope: CoroutineScope,
        taskDaoProvider: Provider<TaskDao>
    ): TaskDatabase {

        return Room.databaseBuilder(
            application.applicationContext,
            TaskDatabase::class.java,
            name: "task_database"
        ).fallbackToDestructiveMigration()
            .addCallback(object : RoomDatabase.Callback() {
                override fun onCreate(db: SupportSQLiteDatabase) {
                    super.onCreate(db)
                    //the same way: https://dev.to/anandpushkar088/experimenting-with-dagger-hilt-3e80
                    scope.launch { this: CoroutineScope
                        populateDatabase(taskDaoProvider.get())
                    }
                }
            })
            .build()

        private suspend fun populateDatabase(taskDao: TaskDao) {
            for (i in 0..10000) {
                taskDao.insertTask(Task(i, title: "Task $i", description: "Description $i", System.currentTimeMillis()))
            }
        }

    }

    @Provides
    @Singleton
    fun provideTaskDao(taskDatabase: TaskDatabase): TaskDao {
        return taskDatabase.taskDao()
    }
}
```

Network module with Hilt

- Here is an example for network module with hilt, we config logging interceptor, okhttpclient, retrofit and create api in this module.
- Then we inject Passenger API to ViewModel with Hilt

```
@HiltViewModel
class PassengersViewModel @Inject constructor(
    private val passengerApi: PassengerApi
) : ViewModel() {
    val passengers = Pager(PagingConfig(pageSize = 10)) {
        PassengersDataSource(passengerApi)
    }.flow.cachedIn(viewModelScope)
}
```

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    private const val BASE_URL = "https://api.instantwebtools.net/v1/"
    @Singleton
    @Provides
    fun providesHttpLoggingInterceptor() = HttpLoggingInterceptor()
        .apply { this: HttpLoggingInterceptor()
            level = HttpLoggingInterceptor.Level.BODY
        }

    @LoggingInterceptorOkHttpClient
    @Singleton
    @Provides
    fun providesOkHttpClient(httpLoggingInterceptor: HttpLoggingInterceptor): OkHttpClient =
        OkHttpClient
            .Builder()
            .addInterceptor(httpLoggingInterceptor)
            .build()

    @Provides
    @Singleton
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .addCallAdapterFactory(RxJava3CallAdapterFactory.create())
            .build()
    }

    @Provides
    @Singleton
    fun providePassengerApi(retrofit: Retrofit): PassengerApi {
        return retrofit.create(PassengerApi::class.java)
    }
}
```

Koin

OPEN MIND - OPEN DOORS

Introduction Koin

- Koin, a lightweight DI framework for Kotlin, leverages several Kotlin features to simplify dependency injection in Android and other Kotlin-based projects
- Here are two examples of how Koin achieves this:
 - Constructor Injection with Parameter Default Values

```
// Service interface and implementation
interface DataService {
    fun fetchData(): List<String>
}

class RemoteDataService : DataService {
    override fun fetchData(): List<String> {
        // Fetch data from a remote server
    }
}

// Koin module setup
val myModule = module {
    // Define the DataService dependency with RemoteDataService as its implementation
    single<DataService> { RemoteDataService() }
}

// Usage of the DataService with constructor injection and default parameter value
class MyViewModel(private val dataService: DataService = get()) {
    fun loadData() {
        val data = dataService.fetchData()
        // Handle the data
    }
}
```

Introduction Koin

- Property Delegation with Koin's 'by inject()'

```
// Koin module setup (same DataService and RemoteDataService as in the previous example)
val myModule = module {
    // Define the DataService dependency with RemoteDataService as its implementation
    single<DataService> { RemoteDataService() }
}

// Usage of the DataService with lazy initialization using Koin's 'by inject()' property
class MyViewModel {
    private val dataService: DataService by inject()

    fun loadData() {
        val data = dataService.fetchData()
        // Handle the data
    }
}
```

Implementation of a Koin Module

- Set up:

```
implementation "io.insert-koin:koin-android:2.2.0" // Replace with the latest version
implementation "io.insert-koin:koin-androidx-scope:2.2.0"
implementation "io.insert-koin:koin-androidx-viewmodel:2.2.0"
```

- Create the Koin Module:

```
import org.koin.dsl.module

val appModule = module {
    // Define your dependencies here
}
```

Implementation of a Koin Module

- Declare Dependencies:

```
import org.koin.dsl.module

val appModule = module {
    single<MyRepository> { MyRepositoryImpl() }
    factory<MyService> { MyService() }
}
```

- Scoped Dependencies:

```
import org.koin.dsl.module

val appModule = module {
    // ... (previous dependencies)

    scope<MyViewModel> {
        MyViewModel(get())
    }
}
```

Implementation of a Koin Module

- Integration into the Application:

```
import org.koin.android.ext.koin.androidContext
import org.koin.core.context.startKoin

class MyApplication : Application() {
    override fun onCreate() {
        super.onCreate()

        val myModules = listOf(appModule)

        startKoin {
            androidContext(this@MyApplication)
            modules(myModules)
        }
    }
}
```

Implementation of a Koin Module

- Inject Dependencies:

```
import org.koin.android.ext.android.inject

class MyActivity : AppCompatActivity() {

    private val myRepository: MyRepository by inject()
    private val myViewModel: MyViewModel by viewModel()

    // ... (rest of the code)
}
```

- Clean up koin

```
class MyApplication : Application() {
    // ... (previous code)

    override fun onTerminate() {
        super.onTerminate()
        stopKoin()
    }
}
```



Thank You !

OPEN MIND - OPEN DOORS